01AL0501 – INTELLIGENT WEB DESIGNING

Practical File

Experiment 2 and Experiment 3

Student name	LUBEGA MAHAD ISHAAQ
Enrollment number	92400198030
Department	BS – AI ML & DS, Faculty of Technology
University	Marwadi University, Rajkot

No.	Title	Folder in repository
2	Build a responsive UI that dynamically updates text based on user input using JavaScript	experiment-2/
3	Create an interactive Line and Bar chart dashboard using Chart.js to display dataset statistics	experiment-3/

GitHub repository: https://github.com/mahadishaaqlubega/iwd-practicals

Live demo (GitHub Pages): https://mahadishaaqlubega.github.io/iwd-practicals/

Experiment 2: https://mahadishaaqlubega.github.io/iwd-practicals/experiment-2/
Experiment 3: https://mahadishaaqlubega.github.io/iwd-practicals/experiment-3/

Experiment 2: Build a responsive UI that dynamically updates text based on user input using JavaScript.

Aim

To build a responsive web interface that dynamically updates text based on user input using JavaScript.

Objectives

- Learn DOM manipulation.
- Capture user input using form elements.
- Update webpage content dynamically.
- Improve user interaction through real-time feedback.

Tools Required

- HTML5
- CSS3
- JavaScript
- Visual Studio Code

Procedure

1. Create a responsive webpage using HTML and CSS.
2. Add input elements such as text boxes and buttons.
3. Write JavaScript to capture user input.
4. Update headings, paragraphs, or labels dynamically.
5. Validate the input and provide instant feedback.
6. Test the application on different screen sizes.

Implementation

I made a page called *Lab Pass Studio*. The left side is a form and the right side is a lab pass, a row of text statistics and a short change log. None of the text on the right is written in the HTML file. JavaScript reads the form and writes it there each time a key is pressed. The design uses a printed-page style: a serif font for headings and numbers, a plain sans-serif for labels, thin rules instead of boxes, and one accent colour.

The page uses `index.html` for the structure, `style.css` for its own layout and `script.js` for the behaviour. Fonts, colours and the header are shared with Experiment 3 through `assets/css/base.css`. This is how each step of the procedure was done:

1. *Responsive layout.* The form and the preview sit in a CSS grid with two columns and a thin rule between them. A media query at 960 px changes the grid to one column, and another at

560 px stacks the department and semester fields and makes the pass more compact. The headline size uses `clamp()` so it shrinks on phones.

2. *Input elements.* Text boxes for name, enrollment number and headline, two drop-down lists, a textarea for a short bio, five accent colour buttons, a three-way name style switch, and Issue pass and Clear buttons.
3. *Capturing input.* One `input` event listener is attached to the whole form instead of one per field. The handler reads `e.target.getAttribute("name")` to find out which field changed.
4. *Updating the page.* A function called `render()` copies the values into the large headline (it greets the user by first name and the time of day), the paragraph under it, the pass, the initials box, the character counter, the completeness meter and the word count. A second function draws barcode bars on the pass from the digits of the enrollment number using SVG `<rect>` elements. Text is set with `textContent`, so anything typed is shown as plain text and never run as HTML. Each change is also added to a change log that keeps the last five edits.
5. *Validation.* Each field has a small rule function that returns an error message or an empty string. The hint under the field turns green or red while typing. The name only allows letters and spaces, the enrollment box removes anything that is not a digit and asks for exactly 11 digits, and the department is required. On Save, every field is checked and the cursor jumps to the first wrong one.
6. *Testing.* I tested the page in Chrome at 1440 px, 768 px and 390 px wide (phone emulation in DevTools), in both light and dark theme.

Problem I ran into

In my first version the change handler read `e.target.name`. When the event came from the form itself, `form.name` did not give a string. It returned the input element whose name is "name", because a form exposes its fields as properties. The script then crashed with "Cannot read properties of undefined". Using `getAttribute("name")` and checking that the key exists in the `inputs` object fixed it.

Program Code

experiment-2/index.html

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab Pass Studio | IWD Experiment 2</title>
  <meta name="description" content="Experiment 2: a responsive interface that rewrites page text as
the user types.">
  <link rel="stylesheet" href="../assets/css/base.css">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="masthead">
    <div class="masthead-row">
      <a class="wordmark" href="../index.html"><b>IWD Lab</b><span>Experiment No. 02</span></a>
```

```
      <div class="mast-meta">
        <span class="hide-sm" id="clock"> </span>
        <button class="text-btn" id="themeBtn" type="button">Dark mode</button>
      </div>
    </div>
  </header>

  <main class="page">

    <!-- headline: rewritten by JavaScript as the user types -->
    <section class="hero">
      <div class="hero-text">
        <p class="kicker">Live text binding</p>
        <h1 class="display" id="greeting">Good day, <em>guest</em>.</h1>
        <p class="deck" id="deck">Fill in the form and the pass on the right is written for you, one
keystroke at a time.</p>
      </div>
      <div class="meter" aria-live="polite">
        <span class="meter-label">Pass complete</span>
        <span class="meter-value"><span id="pctValue">0</span><small>%</small></span>
        <div class="meter-track"><div class="meter-fill" id="pctFill"></div></div>
        <span class="meter-note" id="pctNote">3 required fields left</span>
      </div>
    </section>

    <section class="workspace">

      <!-- left column: the form -->
      <form class="form-col" id="profileForm" novalidate>
        <div class="sec-label"><b>01</b> Your details</div>

        <div class="field" data-field="name">
          <label for="name"><span class="n">a</span>Full name <i>required</i></label>
          <input type="text" id="name" name="name" placeholder="Lubega Mahad Ishaaq"
autocomplete="name" maxlength="40">
          <p class="hint" data-default="Letters and spaces, 3 to 40 characters">Letters and spaces, 3
to 40 characters</p>
        </div>

        <div class="field" data-field="enrollment">
          <label for="enrollment"><span class="n">b</span>Enrollment number <i>required</i></label>
          <input type="text" id="enrollment" name="enrollment" placeholder="00000000000"
inputmode="numeric" maxlength="11">
          <p class="hint" data-default="Exactly 11 digits">Exactly 11 digits</p>
        </div>

        <div class="pair">
          <div class="field" data-field="department">
            <label for="department"><span class="n">c</span>Department <i>required</i></label>
            <select id="department" name="department">
              <option value="">Choose</option>
              <option>BS - AI ML &amp;amp; DS</option>
              <option>Computer Engineering</option>
              <option>Information Technology</option>
              <option>Electronics &amp;amp; Communication</option>
            </select>
            <p class="hint" data-default="Pick one">Pick one</p>
          </div>

          <div class="field" data-field="semester">
            <label for="semester"><span class="n">d</span>Semester</label>
            <select id="semester" name="semester">
              <option value="">Choose</option>
              <option>Semester 3</option>
              <option>Semester 4</option>
              <option>Semester 5</option>
              <option>Semester 6</option>
            </select>
```

```
            <p class="hint" data-default="Optional">Optional</p>
          </div>
        </div>

        <div class="field" data-field="headline">
          <label for="headline"><span class="n">e</span>Headline</label>
          <input type="text" id="headline" name="headline" placeholder="Web developer, AI/ML student"
maxlength="50">
          <p class="hint" data-default="Optional, up to 50 characters">Optional, up to 50
characters</p>
        </div>

        <div class="field" data-field="about">
          <label for="about"><span class="n">f</span>About <span class="counter"
id="aboutCounter">0/160</span></label>
          <textarea id="about" name="about" rows="3" maxlength="160" placeholder="Two lines about
what you are working on."></textarea>
          <p class="hint" data-default="Optional, up to 160 characters">Optional, up to 160
characters</p>
        </div>

        <div class="options">
          <div>
            <span class="opt-label">Accent</span>
            <div class="swatches" id="swatches" role="radiogroup" aria-label="Accent colour">
              <button type="button" class="swatch active" data-color="#c8442c" data-name="Vermillion"
style="--c:#c8442c" aria-label="Vermillion"></button>
              <button type="button" class="swatch" data-color="#1f4e79" data-name="Navy" style="--
c:#1f4e79" aria-label="Navy"></button>
              <button type="button" class="swatch" data-color="#2f6b47" data-name="Forest" style="--
c:#2f6b47" aria-label="Forest"></button>
              <button type="button" class="swatch" data-color="#a87b16" data-name="Ochre" style="--
c:#a87b16" aria-label="Ochre"></button>
              <button type="button" class="swatch" data-color="#171614" data-name="Ink" style="--
c:#171614" aria-label="Ink"></button>
            </div>
          </div>
          <div>
            <span class="opt-label">Name style</span>
            <div class="tabs" id="caseControl">
              <button type="button" data-case="normal" class="active">As typed</button>
              <button type="button" data-case="title">Title Case</button>
              <button type="button" data-case="upper">Capitals</button>
            </div>
          </div>
        </div>

        <div class="actions">
          <button type="submit" class="btn">Issue pass</button>
          <button type="button" class="btn ghost" id="resetBtn">Clear</button>
        </div>
      </form>

      <!-- right column: everything here is written by script.js -->
      <aside class="preview-col">
        <div class="sec-label"><b>02</b> Preview</div>

        <article class="pass" id="pass">
          <header class="pass-top">
            <span>IWD Lab Pass</span>
            <span id="pvYear">2026</span>
          </header>

          <div class="pass-main">
            <div class="monogram" id="pvInitials">?</div>
            <div class="pass-id">
              <h2 id="pvName">Your name</h2>
              <p id="pvHeadline">Your headline</p>
```

```
            </div>
          </div>

          <dl class="pass-meta">
            <div><dt>Enrollment</dt><dd id="pvEnroll">&mdash;</dd></div>
            <div><dt>Department</dt><dd id="pvDept">&mdash;</dd></div>
            <div><dt>Semester</dt><dd id="pvSem">&mdash;</dd></div>
          </dl>

          <p class="pass-about" id="pvAbout">Whatever you write in the About box appears here.</p>

          <footer class="pass-foot">
            <svg class="barcode" id="barcode" viewBox="0 0 220 44" preserveAspectRatio="none" aria-
hidden="true"></svg>
            <span class="code-num" id="pvCode">00000000000</span>
          </footer>

          <span class="stamp" id="stamp">Issued</span>
        </article>

        <div class="figures">
          <div><span id="stChars">0</span><small>Characters</small></div>
          <div><span id="stWords">0</span><small>Words</small></div>
          <div><span id="stRead">0s</span><small>Read time</small></div>
        </div>

        <div class="log">
          <div class="sec-label"><b>03</b> Change log</div>
          <ol id="log"><li class="empty">Nothing typed yet.</li></ol>
        </div>
      </aside>

    </section>
  </main>

  <div class="toast" id="toast" role="status" aria-live="polite"></div>

  <footer class="colophon">
    <span>Lubega Mahad Ishaaq &middot; 92400198030</span>
    <span>01AL0501 Intelligent Web Designing</span>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

experiment-2/script.js

```
// Experiment 2 - Lab Pass Studio
// Reads what the user types and rewrites parts of the page straight away.

const $ = (id) => document.getElementById(id);

const form = $("profileForm");
const inputs = {
  name: $("name"),
  enrollment: $("enrollment"),
  department: $("department"),
  semester: $("semester"),
  headline: $("headline"),
  about: $("about"),
};
const labels = {
  name: "Full name", enrollment: "Enrollment number", department: "Department",
  semester: "Semester", headline: "Headline", about: "About",
};
const required = ["name", "enrollment", "department"];
```

```
let nameCase = "normal";

// ---------- validation rules ----------
// each rule returns an error message, or "" when the value is fine
const rules = {
  name(v) {
    if (!v) return "Name is required";
    if (!/^[A-Za-z ]+$/.test(v)) return "Only letters and spaces are allowed";
    if (v.replace(/\s/g, "").length < 3) return "Too short, at least 3 letters";
    return "";
  },
  enrollment(v) {
    if (!v) return "Enrollment number is required";
    if (v.length !== 11) return `${v.length} of 11 digits entered`;
    return "";
  },
  department: (v) => (v ? "" : "Choose a department"),
  semester: () => "",
  headline: (v) => (v.length > 50 ? "Keep it under 50 characters" : ""),
  about: (v) => (v.length > 160 ? "Maximum is 160 characters" : ""),
};

const okText = {
  name: "Looks good", enrollment: "Valid enrollment number", department: "Selected",
  semester: "Selected", headline: "Added to the pass", about: "Added to the pass",
};

// ---------- helpers ----------
const toTitle = (s) => s.toLowerCase().replace(/\b[a-z]/g, (c) => c.toUpperCase());

function styledName(raw) {
  const clean = raw.trim().replace(/\s+/g, " ");
  if (nameCase === "upper") return clean.toUpperCase();
  if (nameCase === "title") return toTitle(clean);
  return clean;
}

function initials(name) {
  const parts = name.trim().split(/\s+/).filter(Boolean);
  if (!parts.length) return "?";
  return (parts[0][0] + (parts.length > 1 ? parts.at(-1)[0] : "")).toUpperCase();
}

function timeGreeting() {
  const h = new Date().getHours();
  return h < 12 ? "Good morning" : h < 17 ? "Good afternoon" : "Good evening";
}

// write text into an element and flash it, but only if it actually changed
function setText(el, value, fallback) {
  const next = value || fallback;
  if (el.textContent === next) return;
  el.textContent = next;
  el.classList.toggle("placeholder", !value);
  el.classList.remove("flash");
  void el.offsetWidth; // restart the animation
  el.classList.add("flash");
}

// draws simple barcode-style bars from the digits of the enrollment number
function drawBarcode(digits) {
  const svg = $("barcode");
  const src = (digits || "00000000000").padEnd(11, "0");
  let x = 0, bars = "";
  for (const ch of src) {
    const d = Number(ch);
    // every digit gives three bars of different widths
```

```
    const widths = [1 + (d % 3), 1 + ((d + 1) % 2), 2 + (d % 2)];
    for (const w of widths) {
      bars += `<rect x="${x}" y="0" width="${w * 1.6}" height="44" fill="currentColor"/>`;
      x += w * 1.6 + 2.2;
    }
  }
  svg.setAttribute("viewBox", `0 0 ${x} 44`);
  svg.innerHTML = bars;
}

// ---------- show the result of one field's rule ----------
const touched = new Set();

function checkField(key) {
  const input = inputs[key];
  const box = input.closest(".field");
  const hint = box.querySelector(".hint");
  const value = input.value.trim();
  const error = rules[key](value);

  box.classList.remove("valid", "invalid");
  if (!touched.has(key) || (!value && !required.includes(key))) {
    hint.textContent = hint.dataset.default;
    return !error;
  }
  box.classList.add(error ? "invalid" : "valid");
  hint.textContent = error || okText[key];
  return !error;
}

// ---------- main render: copy the form values into the page ----------
function render() {
  const v = {};
  for (const key in inputs) v[key] = inputs[key].value.trim();

  const shown = styledName(v.name);
  const first = shown.split(" ")[0];

  // headline and deck
  const greet = $("greeting");
  greet.textContent = `${timeGreeting()}, `;
  const em = document.createElement("em");
  em.textContent = first || "guest";
  greet.append(em, ".");

  $("deck").textContent = first
    ? `${first}, your pass is being written as you type. Finish the three required fields to issue
it.`
    : "Fill in the form and the pass on the right is written for you, one keystroke at a time.";

  // the pass
  setText($("pvName"), shown, "Your name");
  setText($("pvHeadline"), v.headline, "Your headline");
  setText($("pvEnroll"), v.enrollment, "—");
  setText($("pvDept"), v.department, "—");
  setText($("pvSem"), v.semester.replace("Semester ", "Sem "), "—");
  setText($("pvAbout"), v.about, "Whatever you write in the About box appears here.");
  $("pvInitials").textContent = initials(v.name);
  $("pvCode").textContent = (v.enrollment || "").padEnd(11, "0");
  drawBarcode(v.enrollment);

  // about counter
  const len = inputs.about.value.length;
  $("aboutCounter").textContent = `${len}/160`;
  $("aboutCounter").classList.toggle("near", len >= 140);

  // completeness meter, required fields count double
  let score = 0, total = 0;
```

```
  for (const key in inputs) {
    const weight = required.includes(key) ? 2 : 1;
    total += weight;
    if (v[key] && !rules[key](v[key])) score += weight;
  }
  const pct = Math.round((score / total) * 100);
  $("pctValue").textContent = pct;
  $("pctFill").style.width = pct + "%";

  const missing = required.filter((k) => rules[k](v[k])).length;
  $("pctNote").textContent = missing
    ? `${missing} required field${missing > 1 ? "s" : ""} left`
    : "Ready to issue";

  // text statistics
  const all = [v.name, v.headline, v.about].join(" ").trim();
  const words = all ? all.split(/\s+/).length : 0;
  const secs = Math.ceil((words / 200) * 60); // about 200 words a minute
  $("stChars").textContent = all.replace(/\s/g, "").length;
  $("stWords").textContent = words;
  $("stRead").textContent = secs + "s";
}

// ---------- change log ----------
function logChange(text) {
  const list = $("log");
  const empty = list.querySelector(".empty");
  if (empty) empty.remove();

  // typing in the same field updates the top entry instead of adding a new one
  const top = list.firstElementChild;
  const time = new Date().toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-
digit" });
  if (top && top.dataset.text === text) {
    top.querySelector("time").textContent = time;
    return;
  }
  const li = document.createElement("li");
  li.dataset.text = text;
  li.innerHTML = "<span></span><time></time>";
  li.querySelector("span").textContent = text;
  li.querySelector("time").textContent = time;
  list.prepend(li);
  while (list.children.length > 5) list.lastElementChild.remove();
}

// ---------- events ----------
// one listener on the form handles every field (event delegation)
function onFieldChange(e) {
  const key = e.target.getAttribute("name");
  if (!key || !(key in inputs)) return;

  if (key === "enrollment") e.target.value = e.target.value.replace(/\D/g, "");

  touched.add(key);
  checkField(key);
  render();
  $("stamp").classList.remove("on");
  logChange(`${labels[key]} edited`);
}

form.addEventListener("input", onFieldChange);
form.addEventListener("change", (e) => {
  if (e.target.tagName === "SELECT") onFieldChange(e);
});

$("swatches").addEventListener("click", (e) => {
  const btn = e.target.closest(".swatch");
```

```
  if (!btn) return;
  document.querySelectorAll(".swatch").forEach((s) => s.classList.toggle("active", s === btn));
  document.documentElement.style.setProperty("--pass", btn.dataset.color);
  logChange(`Accent set to ${btn.dataset.name}`);
});

$("caseControl").addEventListener("click", (e) => {
  const btn = e.target.closest("button");
  if (!btn) return;
  document.querySelectorAll("#caseControl button").forEach((b) => b.classList.toggle("active", b ===
btn));
  nameCase = btn.dataset.case;
  render();
  logChange(`Name style: ${btn.textContent}`);
});

form.addEventListener("submit", (e) => {
  e.preventDefault();
  Object.keys(inputs).forEach((k) => touched.add(k));
  const allValid = Object.keys(inputs).map(checkField).every(Boolean);

  if (allValid) {
    $("stamp").classList.add("on");
    showToast(`Pass issued to ${styledName(inputs.name.value)}`);
    logChange("Pass issued");
  } else {
    const firstBad = form.querySelector(".field.invalid input, .field.invalid select, .field.invalid
textarea");
    if (firstBad) firstBad.focus();
    showToast("Some fields need fixing first");
  }
});

$("resetBtn").addEventListener("click", () => {
  form.reset();
  touched.clear();
  Object.keys(inputs).forEach(checkField);
  $("stamp").classList.remove("on");
  render();
  logChange("Form cleared");
});

let toastTimer;
function showToast(msg) {
  const t = $("toast");
  t.textContent = msg;
  t.classList.add("show");
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => t.classList.remove("show"), 2600);
}

// ---------- theme (remembered between visits) ----------
function applyTheme(theme) {
  document.documentElement.dataset.theme = theme;
  $("themeBtn").textContent = theme === "dark" ? "Light mode" : "Dark mode";
}
$("themeBtn").addEventListener("click", () => {
  const next = document.documentElement.dataset.theme === "dark" ? "light" : "dark";
  applyTheme(next);
  try { localStorage.setItem("iwd-theme", next); } catch (err) { /* storage blocked */ }
});
try { applyTheme(localStorage.getItem("iwd-theme") || "light"); } catch (err) { applyTheme("light");
}

// ---------- clock in the masthead ----------
function tick() {
  const now = new Date();
  $("clock").textContent = now.toLocaleDateString("en-GB", { weekday: "short", day: "numeric", month:
```

```
"short", year: "numeric" }) +
    "  " + now.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" });
  $("pvYear").textContent = now.getFullYear();
}
tick();
setInterval(tick, 30000);

render();
```

The full `style.css` and the shared `assets/css/base.css` are in the repository. The parts that control the validation messages and the responsive layout are shown below.

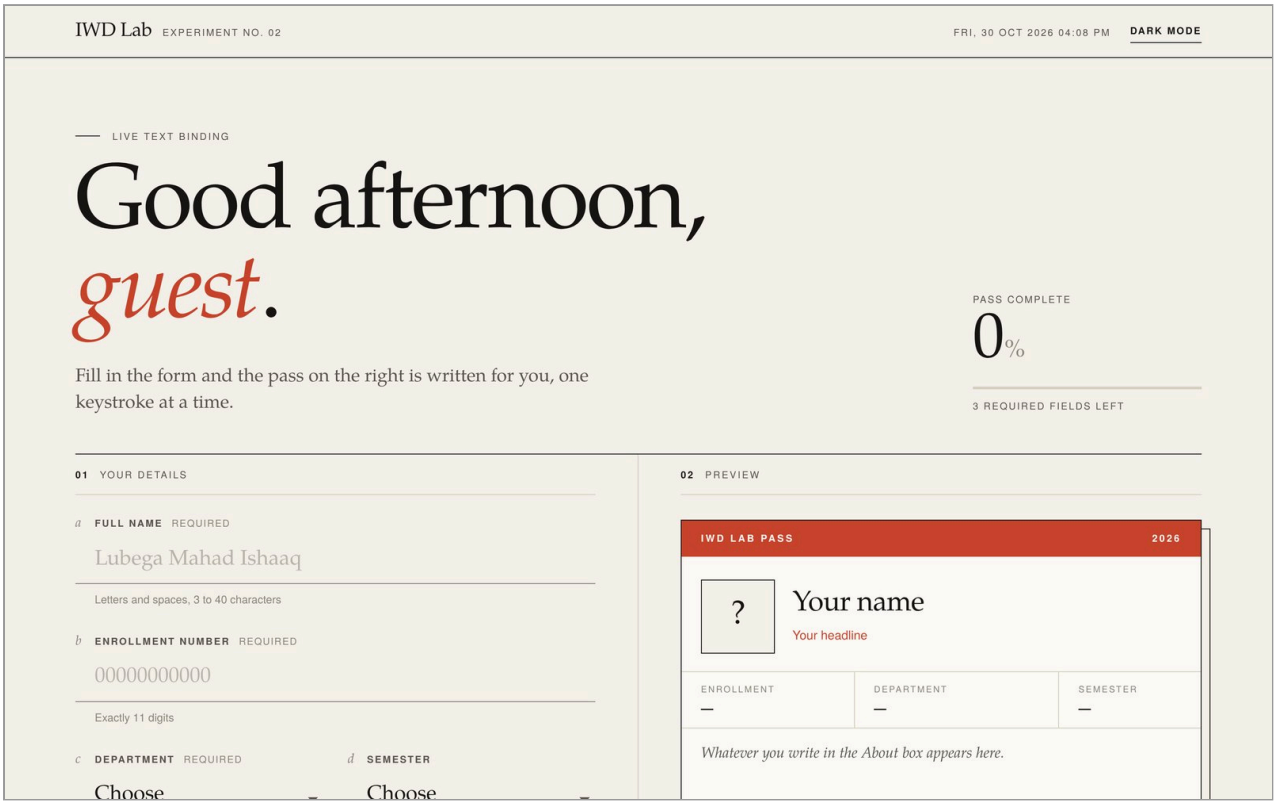
experiment-2/style.css

```
.field.valid .hint { color: var(--good); }
.field.valid .hint::before { content: "\2713"; }
.field.invalid .hint { color: var(--bad); }
.field.invalid .hint::before { content: "\2715"; }
.field.invalid input, .field.invalid select, .field.invalid textarea { border-bottom-color: var(--
bad); box-shadow: 0 1px 0 var(--bad); }
```

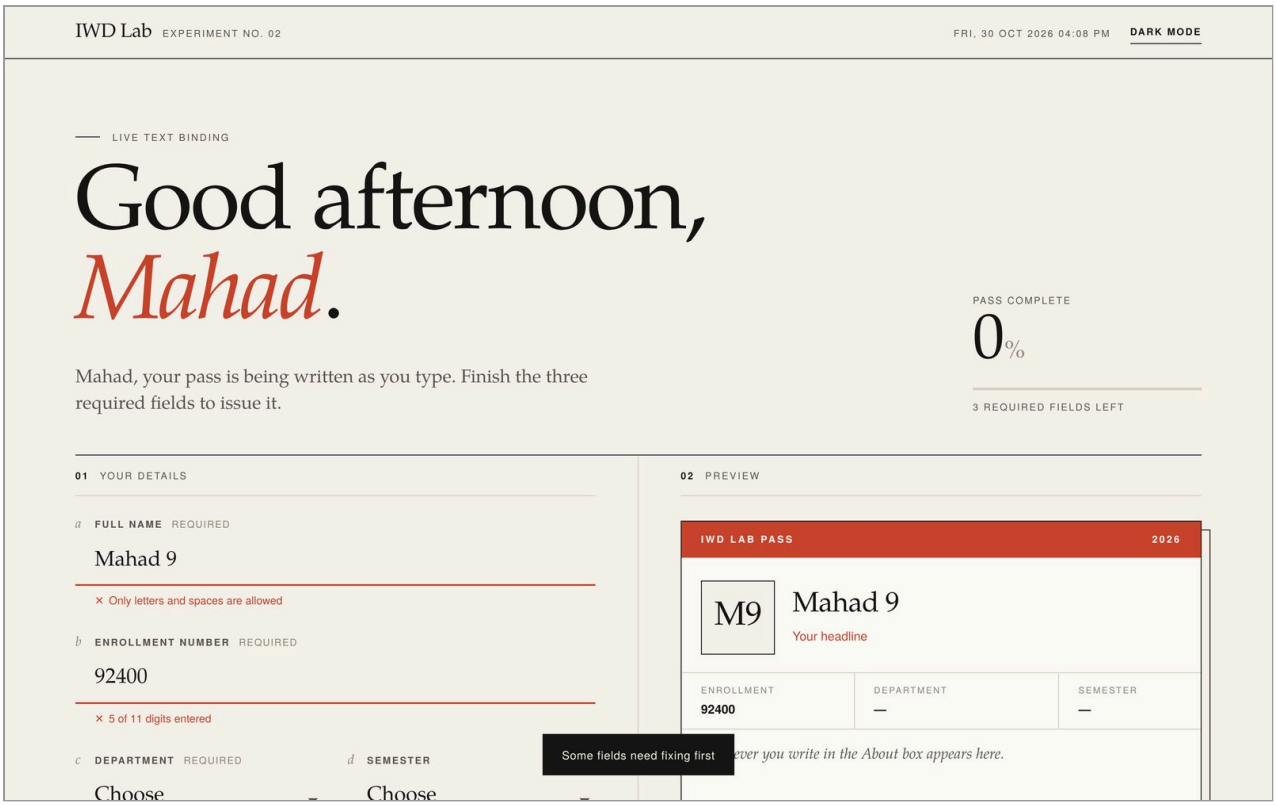
experiment-2/style.css

```
/* ---------- responsive ---------- */
@media (max-width: 960px) {
  .hero { grid-template-columns: 1fr; gap: 28px; }
  .meter { max-width: 320px; }
  .workspace { grid-template-columns: 1fr; }
  .form-col { padding: 0; border-right: 0; }
  .preview-col { padding: 40px 0 0; }
}
@media (max-width: 560px) {
  .pair { grid-template-columns: 1fr; gap: 0; }
  input, select { font-size: 20px; }
  .pass-meta { grid-template-columns: 1fr 1fr; }
  .pass-meta div:nth-child(2) { border-right: 0; }
  .pass-meta div:last-child { grid-column: span 2; border-top: 1px solid var(--rule); }
  .pass-main { padding: 22px 18px 18px; }
  .monogram { width: 64px; height: 64px; font-size: 28px; }
  .pass-id h2 { font-size: 26px; }
  .figures span { font-size: 32px; }
  .actions .btn { flex: 1; }
  .meter-value { font-size: 56px; }
}
```

Output



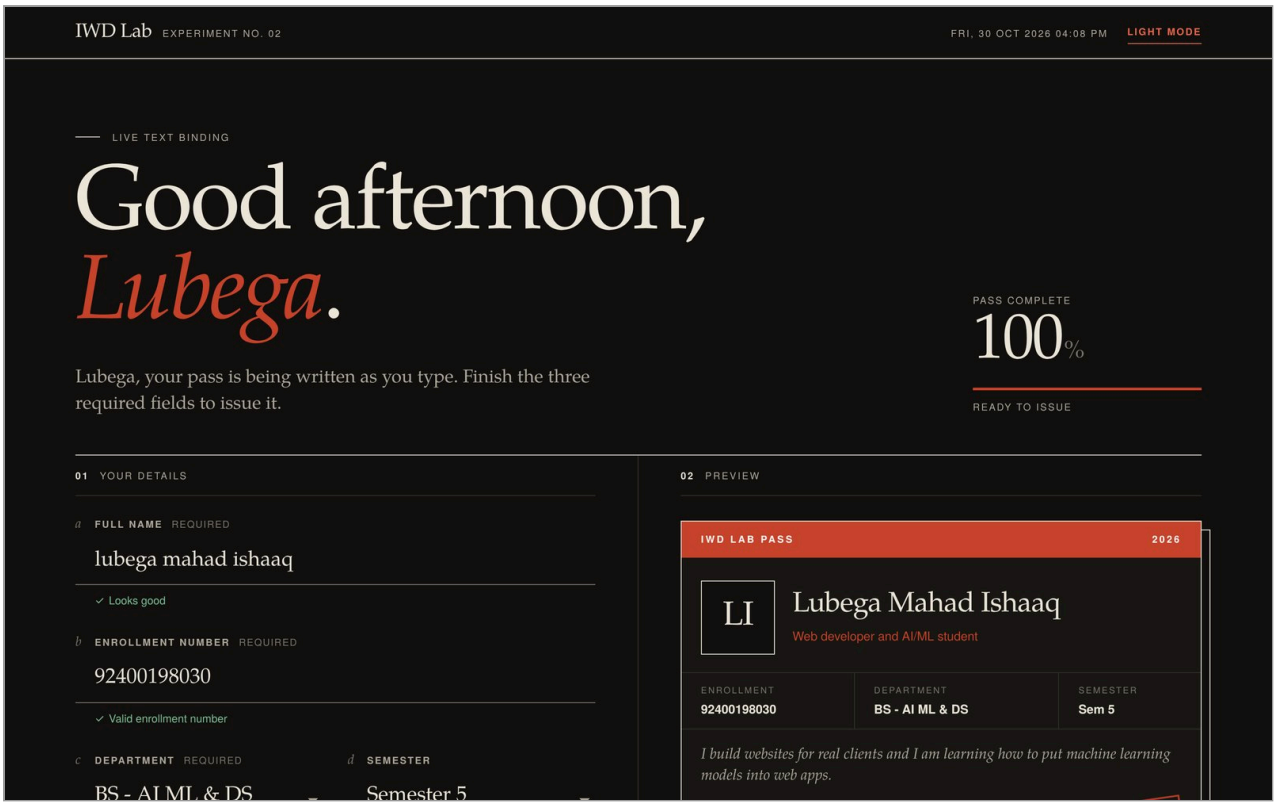
Page on first load. The pass shows placeholder text and the meter is at 0%.



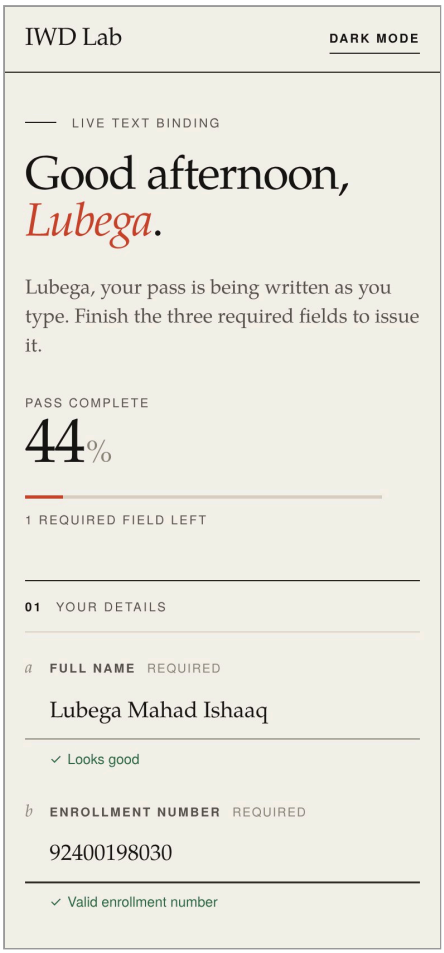
Wrong input: a digit in the name, only 5 digits in the enrollment number and no department. Each field shows its own message, and the headline and pass still update.



All fields filled, Title Case and the navy accent selected. After Issue pass, the pass is stamped and the change log records it.



Dark theme. The headline, meter and pass keep updating in the same way.



The same page at 390 px (phone) width. The two columns stack into one.

Result

The page updates the headline, the paragraph, the pass and the counters on every keystroke without reloading. Wrong values are marked as soon as they are typed, and the Issue pass button only accepts the form when the three required fields are valid. The layout works at desktop, tablet and phone widths.

Conclusion

This experiment showed me how the DOM connects a form to the rest of the page. One event listener on the parent form was enough to handle every field, and keeping all page updates inside one `render()` function made the code easier to follow. I also learned to use `textContent` for user text and to validate while the user types instead of only on submit.

Experiment 3: Visualization: Create an interactive Line and Bar chart dashboard using Chart.js to display dataset statistics.

Aim

To create an interactive dashboard using Chart.js for displaying dataset statistics through Line and Bar charts.

Objectives

- Learn Chart.js fundamentals.
- Visualize dataset statistics using different chart types.
- Create interactive and responsive charts.
- Compare multiple datasets visually.

Tools Required

- HTML5
- CSS3
- JavaScript
- Chart.js (version 4.4.1, loaded from the jsDelivr CDN)
- Visual Studio Code
- GitHub and GitHub Pages for hosting

Procedure

1. Create an HTML page with two <canvas> elements.
2. Include the Chart.js library.
3. Define a sample dataset in JavaScript.
4. Create a Line Chart to display trends over time.
5. Create a Bar Chart to compare different categories.
6. Add chart options such as titles, legends, and tooltips.
7. Test responsiveness by resizing the browser window.

Sample Dataset

Month	Sales	Orders
Jan	120	50
Feb	150	60
Mar	180	75
Apr	170	68

May	210	85
Jun	240	95

Implementation

The dashboard is called *The Sales Ledger*. It uses the dataset above and has four parts: a strip of four headline figures, the two charts, a statistics table and a table of monthly entries. It follows the same printed-page design as Experiment 2, with black and one accent colour for the two data series.

1. *Two canvas elements.* `#lineChart` and `#barChart` are each placed inside a `div.canvas-wrap` with a fixed height. Chart.js needs this when `maintainAspectRatio` is false, otherwise the chart keeps growing.
2. *Including Chart.js.* A single `<script>` tag in the head loads `chart.umd.min.js` from jsDelivr. If it fails to load, the page shows a message instead of empty boxes.
3. *Dataset.* The data is stored in an object `ORIGINAL` with three arrays: `months`, `sales` and `orders`. A copy is made with `structuredClone()` so the Reset button can restore it.
4. *Line chart.* Sales and orders are plotted against months. Sales uses the left y-axis and orders the right one (`yAxisID: "y1"`), because orders are much smaller numbers and would look flat on the same scale. The sales line has a light fill under it and the orders line is dashed. A Curved / Straight switch changes the `tension` option.
5. *Bar chart.* Both series are shown side by side for each month. A Vertical / Horizontal switch rebuilds the chart with `indexAxis: "y"`.
6. *Titles, legends and tooltips.* Both charts have a title and a legend at the bottom. Clicking a legend item hides that series. The line chart tooltip shows both values for the month and an extra line with sales per order.
7. *Responsiveness.* Both charts use `responsive: true`. Under 1024 px the charts and tables move into one column, and the headline figures change from four in a row to two.

Statistics shown

The statistics table is worked out in JavaScript from the selected period: total, mean, median, highest and lowest value with the month, range, and standard deviation (population formula). The correlation between sales and orders is shown above the table. For the six original months the values are:

Statistic	Sales	Orders
Total	1070	433
Mean	178.33	72.17
Median	175.0	71.5

Highest / Lowest	240 (Jun) / 120 (Jan)	95 (Jun) / 50 (Jan)
Standard deviation	38.91	15.00
Correlation r	0.997	

The only month where sales went down is April (from 180 to 170, a 5.6% drop). Sales doubled from January to June, a growth of 100%. An r value of 0.997 means orders rise and fall almost exactly with sales.

Extra features

A period filter shows Jan to Mar, Apr to Jun or all months, and every figure, chart and table follows it. The "Add the next month" form adds a new row after checking that both numbers are whole, positive and that orders are not more than sales. The "Start live feed" button adds a new month every 2.5 seconds to simulate live data, up to a window of 12 months. There is also a dark theme.

Problem I ran into

Chart.js reads the colours only when a chart is created, so switching to the dark theme left the charts with dark lines on a dark background. I fixed this by reading the colours from CSS variables with `getComputedStyle` and destroying and rebuilding both charts when the theme changes.

Program Code

experiment-3/index.html

```
<!DOCTYPE html>
<html lang="en" data-theme="light">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sales Ledger | IWD Experiment 3</title>
  <meta name="description" content="Experiment 3: interactive line and bar chart dashboard built with
Chart.js.">
  <link rel="stylesheet" href="../assets/css/base.css">
  <link rel="stylesheet" href="style.css">
  <!-- Chart.js from the jsDelivr CDN -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
</head>
<body>

  <header class="masthead">
    <div class="masthead-row">
      <a class="wordmark" href="../index.html"><b>IWD Lab</b><span>Experiment No. 03</span></a>
      <div class="mast-meta">
        <span class="live" id="liveBadge">Static data</span>
        <button class="text-btn" id="themeBtn" type="button">Dark mode</button>
      </div>
    </div>
  </header>

  <main class="page">

    <section class="hero">
      <div>
        <p class="kicker">Chart.js dashboard</p>
```

```
        <h1 class="display">The Sales <em>Ledger</em></h1>
      </div>
      <div class="hero-side">
        <p class="deck">Six months of sales and orders. The line shows how the numbers moved, the
bars compare each month, and the tables hold the figures behind both.</p>
        <div class="period">
          <span class="opt-label">Period</span>
          <div class="tabs" id="rangeControl">
            <button type="button" data-range="q1">Jan &ndash; Mar</button>
            <button type="button" data-range="q2">Apr &ndash; Jun</button>
            <button type="button" data-range="all" class="active">All months</button>
          </div>
        </div>
      </div>
    </section>

    <!-- headline figures, filled by script.js -->
    <section class="kpis">
      <div class="kpi">
        <span class="kpi-label">Total sales</span>
        <strong id="kpiSales">0</strong>
        <span class="kpi-sub" id="kpiSalesSub"> </span>
      </div>
      <div class="kpi">
        <span class="kpi-label">Total orders</span>
        <strong id="kpiOrders">0</strong>
        <span class="kpi-sub" id="kpiOrdersSub"> </span>
      </div>
      <div class="kpi">
        <span class="kpi-label">Sales per order</span>
        <strong id="kpiRatio">0</strong>
        <span class="kpi-sub">across the period</span>
      </div>
      <div class="kpi">
        <span class="kpi-label">Sales growth</span>
        <strong id="kpiGrowth">0%</strong>
        <span class="kpi-sub" id="kpiGrowthSub"> </span>
      </div>
    </section>

    <section class="charts">
      <article class="chart-block">
        <div class="block-head">
          <div>
            <p class="sec-num">01 &middot; Line chart</p>
            <h2>How sales and orders moved</h2>
            <p class="sub">Sales on the left scale, orders on the right</p>
          </div>
          <div class="tabs" id="lineMode">
            <button type="button" data-tension="0.4" class="active">Curved</button>
            <button type="button" data-tension="0">Straight</button>
          </div>
        </div>
        <div class="canvas-wrap"><canvas id="lineChart" role="img" aria-label="Line chart of monthly
sales and orders"></canvas></div>
      </article>

      <article class="chart-block">
        <div class="block-head">
          <div>
            <p class="sec-num">02 &middot; Bar chart</p>
            <h2>Month against month</h2>
            <p class="sub">Both series side by side for each month</p>
          </div>
          <div class="tabs" id="barMode">
            <button type="button" data-mode="grouped" class="active">Vertical</button>
            <button type="button" data-mode="horizontal">Horizontal</button>
          </div>
```

```
        </div>
        <div class="canvas-wrap"><canvas id="barChart" role="img" aria-label="Bar chart comparing
sales and orders by month"></canvas></div>
      </article>
    </section>

    <section class="tables">
      <article>
        <div class="block-head">
          <div>
            <p class="sec-num">03 &middot; Statistics</p>
            <h2>The numbers behind the charts</h2>
            <p class="sub" id="statsCaption"> </p>
          </div>
        </div>
        <table class="ledger">
          <thead><tr><th>Measure</th><th>Sales</th><th>Orders</th></tr></thead>
          <tbody id="statsBody"></tbody>
        </table>
      </article>

      <article>
        <div class="block-head">
          <div>
            <p class="sec-num">04 &middot; Data</p>
            <h2>Monthly entries</h2>
            <p class="sub">Change in sales against the month before</p>
          </div>
        </div>
        <table class="ledger">
          <thead><tr><th>Month</th><th>Sales</th><th>Orders</th><th>Change</th></tr></thead>
          <tbody id="dataBody"></tbody>
        </table>

        <form class="add-form" id="addForm" novalidate>
          <span class="opt-label">Add the next month</span>
          <div class="add-row">
            <label><span>Sales</span><input type="number" id="newSales" min="1" max="9999"
placeholder="0"></label>
            <label><span>Orders</span><input type="number" id="newOrders" min="1" max="9999"
placeholder="0"></label>
            <button type="submit" class="btn">Add row</button>
          </div>
          <p class="form-msg" id="formMsg"> </p>
          <div class="add-row links">
            <button type="button" class="text-btn" id="liveBtn">Start live feed</button>
            <button type="button" class="text-btn" id="resetBtn">Reset data</button>
          </div>
        </form>
      </article>
    </section>
  </main>

  <footer class="colophon">
    <span>Lubega Mahad Ishaaq &middot; 92400198030</span>
    <span>01AL0501 Intelligent Web Designing</span>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

experiment-3/script.js

```
// Experiment 3 - The Sales Ledger
// Line and bar charts built with Chart.js, plus statistics worked out from the same dataset.
```

```
const $ = (id) => document.getElementById(id);

// ---------- dataset (from the practical sheet) ----------
const ORIGINAL = {
  months: ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
  sales:  [120, 150, 180, 170, 210, 240],
  orders: [50, 60, 75, 68, 85, 95],
};
const MONTH_NAMES = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov",
"Dec"];

// working copy, so "Reset data" can bring the original back
let data = structuredClone(ORIGINAL);
let range = "all";
let newRowIndex = -1;

// ---------- statistics helpers ----------
const sum = (a) => a.reduce((t, x) => t + x, 0);
const mean = (a) => sum(a) / a.length;
function median(a) {
  const s = [...a].sort((x, y) => x - y);
  const mid = Math.floor(s.length / 2);
  return s.length % 2 ? s[mid] : (s[mid - 1] + s[mid]) / 2;
}
function stdDev(a) {
  const m = mean(a);
  return Math.sqrt(mean(a.map((x) => (x - m) ** 2)));
}
function correlation(a, b) {
  const ma = mean(a), mb = mean(b);
  let top = 0, da = 0, db = 0;
  for (let i = 0; i < a.length; i++) {
    top += (a[i] - ma) * (b[i] - mb);
    da += (a[i] - ma) ** 2;
    db += (b[i] - mb) ** 2;
  }
  return top / Math.sqrt(da * db);
}
const fmt = (n, d = 0) => n.toLocaleString("en-IN", { minimumFractionDigits: d,
maximumFractionDigits: d });

// months for the selected period
function currentSlice() {
  let start = 0, end = data.months.length;
  if (range === "q1") end = 3;
  if (range === "q2") { start = 3; end = 6; }
  return {
    start,
    months: data.months.slice(start, end),
    sales: data.sales.slice(start, end),
    orders: data.orders.slice(start, end),
  };
}

// ---------- chart colours come from CSS, so they follow the theme ----------
function colors() {
  const css = getComputedStyle(document.documentElement);
  const get = (v) => css.getPropertyValue(v).trim();
  return { s1: get("--s1"), s2: get("--s2"), ink: get("--ink"), ink2: get("--ink-2"), ink3: get("--
ink-3"), rule: get("--rule"), paper: get("--paper"), card: get("--card") };
}

function applyChartDefaults() {
  const c = colors();
  Chart.defaults.font.family = "Heros, 'Helvetica Neue', Helvetica, Arial, sans-serif";
  Chart.defaults.font.size = 12;
  Chart.defaults.color = c.ink2;
  Chart.defaults.borderColor = c.rule;
```

```
}

const titleFont = { family: "Pagella, Palatino, Georgia, serif", size: 15, weight: "normal" };

function tooltipStyle(c) {
  return {
    backgroundColor: c.card,
    borderColor: c.ink,
    borderWidth: 1,
    titleColor: c.ink,
    bodyColor: c.ink,
    footerColor: c.ink2,
    titleFont: { ...titleFont, size: 16 },
    footerFont: { weight: "normal", size: 12 },
    padding: 12,
    cornerRadius: 0,
    boxPadding: 5,
    usePointStyle: true,
  };
}

function legendStyle(pointStyle) {
  return { position: "bottom", align: "start", labels: { usePointStyle: true, pointStyle, boxWidth:
22, padding: 18 } };
}

// light fill under the sales line
function salesFill(context) {
  const { ctx, chartArea } = context.chart;
  if (!chartArea) return "transparent";
  const g = ctx.createLinearGradient(0, chartArea.top, 0, chartArea.bottom);
  const base = colors().s1;
  g.addColorStop(0, base + "1f");
  g.addColorStop(1, base + "00");
  return g;
}

// ---------- line chart ----------
let lineChart, barChart;

function buildLineChart(tension = 0.4) {
  const c = colors();
  const s = currentSlice();
  const point = { pointRadius: 3.5, pointHoverRadius: 6, pointBackgroundColor: c.paper,
pointBorderWidth: 2, tension };

  lineChart = new Chart($("lineChart"), {
    type: "line",
    data: {
      labels: s.months,
      datasets: [
        { label: "Sales", data: s.sales, borderColor: c.s1, backgroundColor: salesFill, fill: true,
borderWidth: 2.5, yAxisID: "y", ...point },
        { label: "Orders", data: s.orders, borderColor: c.s2, backgroundColor: c.s2, borderWidth:
2.5, borderDash: [5, 4], yAxisID: "y1", ...point },
      ],
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      interaction: { mode: "index", intersect: false },
      plugins: {
        title: { display: true, text: "Monthly sales and orders", align: "start", color: c.ink, font:
titleFont, padding: { bottom: 16 } },
        legend: legendStyle("line"),
        tooltip: {
          ...tooltipStyle(c),
          callbacks: {
```

```
            footer: (items) => {
              const i = items[0].dataIndex;
              const sales = lineChart.data.datasets[0].data[i];
              const orders = lineChart.data.datasets[1].data[i];
              return `Sales per order  ${(sales / orders).toFixed(2)}`;
            },
          },
        },
      },
      scales: {
        x: { grid: { display: false }, border: { color: c.ink } },
        y: { position: "left", grace: "8%", border: { display: false }, grid: { color: c.rule,
drawTicks: false }, ticks: { padding: 8 }, title: { display: true, text: "Sales", color: c.ink3 } },
        y1: { position: "right", grace: "8%", border: { display: false }, grid: { drawOnChartArea:
false, drawTicks: false }, ticks: { padding: 8 }, title: { display: true, text: "Orders", color:
c.ink3 } },
      },
    },
  });
}

// ---------- bar chart ----------
function buildBarChart(horizontal = false) {
  const c = colors();
  const s = currentSlice();
  const valueAxis = { beginAtZero: true, border: { display: false }, grid: { color: c.rule,
drawTicks: false }, ticks: { padding: 8 } };
  const catAxis = { grid: { display: false }, border: { color: c.ink } };

  barChart = new Chart($("barChart"), {
    type: "bar",
    data: {
      labels: s.months,
      datasets: [
        { label: "Sales", data: s.sales, backgroundColor: c.s1, borderRadius: 0, maxBarThickness: 26,
categoryPercentage: 0.62, barPercentage: 0.9 },
        { label: "Orders", data: s.orders, backgroundColor: c.s2, borderRadius: 0, maxBarThickness:
26, categoryPercentage: 0.62, barPercentage: 0.9 },
      ],
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      indexAxis: horizontal ? "y" : "x",
      plugins: {
        title: { display: true, text: "Sales and orders by month", align: "start", color: c.ink,
font: titleFont, padding: { bottom: 16 } },
        legend: legendStyle("rect"),
        tooltip: tooltipStyle(c),
      },
      scales: horizontal ? { x: valueAxis, y: catAxis } : { x: catAxis, y: valueAxis },
    },
  });
}

// ---------- headline figures and tables ----------
function renderNumbers() {
  const s = currentSlice();
  const totalSales = sum(s.sales);
  const totalOrders = sum(s.orders);
  const growth = ((s.sales.at(-1) - s.sales[0]) / s.sales[0]) * 100;

  $("kpiSales").textContent = fmt(totalSales);
  $("kpiSalesSub").textContent = `${fmt(mean(s.sales), 1)} a month on average`;
  $("kpiOrders").textContent = fmt(totalOrders);
  $("kpiOrdersSub").textContent = `${fmt(mean(s.orders), 1)} a month on average`;
  $("kpiRatio").textContent = (totalSales / totalOrders).toFixed(2);
  $("kpiGrowth").textContent = (growth >= 0 ? "+" : "") + growth.toFixed(1) + "%";
```

```
  const gs = $("kpiGrowthSub");
  gs.textContent = `${s.months[0]} to ${s.months.at(-1)}`;
  gs.className = "kpi-sub " + (growth >= 0 ? "up" : "down");

  // statistics table
  const at = (arr, v) => s.months[arr.indexOf(v)];
  const hi = (arr) => Math.max(...arr);
  const lo = (arr) => Math.min(...arr);
  const rows = [
    ["Total", fmt(totalSales), fmt(totalOrders)],
    ["Mean", fmt(mean(s.sales), 2), fmt(mean(s.orders), 2)],
    ["Median", fmt(median(s.sales), 1), fmt(median(s.orders), 1)],
    ["Highest", `${hi(s.sales)}<small>${at(s.sales, hi(s.sales))}</small>`, `${hi(s.orders)}
<small>${at(s.orders, hi(s.orders))}</small>`],
    ["Lowest", `${lo(s.sales)}<small>${at(s.sales, lo(s.sales))}</small>`, `${lo(s.orders)}
<small>${at(s.orders, lo(s.orders))}</small>`],
    ["Range", hi(s.sales) - lo(s.sales), hi(s.orders) - lo(s.orders)],
    ["Standard deviation", fmt(stdDev(s.sales), 2), fmt(stdDev(s.orders), 2)],
  ];
  $("statsBody").innerHTML = rows.map((r) => `<tr><td>${r[0]}</td><td class="num">${r[1]}</td><td
class="num">${r[2]}</td></tr>`).join("");

  const r = correlation(s.sales, s.orders);
  const label = range === "all" ? "All months" : `${s.months[0]} to ${s.months.at(-1)}`;
  $("statsCaption").textContent = `${label}, ${s.months.length} entries. Correlation between sales
and orders: r = ${r.toFixed(3)}`;

  // raw data with month-on-month change in sales
  $("dataBody").innerHTML = s.months.map((m, i) => {
    const g = s.start + i;
    const prev = data.sales[g - 1];
    let change = '<span class="delta flat">&ndash;</span>';
    if (prev !== undefined) {
      const ch = ((s.sales[i] - prev) / prev) * 100;
      const cls = ch > 0 ? "up" : ch < 0 ? "down" : "flat";
      const arrow = ch > 0 ? "↑" : ch < 0 ? "↓" : "";
      change = `<span class="delta ${cls}">${arrow} ${Math.abs(ch).toFixed(1)}%</span>`;
    }
    const cls = g === newRowIndex ? ' class="new-row"' : "";
    return `<tr${cls}><td>${m}</td><td class="num">${s.sales[i]}</td><td class="num">${s.orders[i]}
</td><td>${change}</td></tr>`;
  }).join("");
}

// send the current slice to both charts
function refresh() {
  const s = currentSlice();
  for (const chart of [lineChart, barChart]) {
    chart.data.labels = s.months;
    chart.data.datasets[0].data = s.sales;
    chart.data.datasets[1].data = s.orders;
    chart.update();
  }
  renderNumbers();
}

// ---------- adding data ----------
function addMonth(sales, orders) {
  const next = MONTH_NAMES[MONTH_NAMES.indexOf(data.months.at(-1)) + 1] || "Jan";
  data.months.push(next);
  data.sales.push(sales);
  data.orders.push(orders);
  // keep at most 12 months on screen
  if (data.months.length > 12) {
    data.months.shift(); data.sales.shift(); data.orders.shift();
  }
  newRowIndex = data.months.length - 1;
  return next;
```

```
}

function message(text, type = "") {
  const msg = $("formMsg");
  msg.textContent = text;
  msg.className = "form-msg " + type;
}

$("addForm").addEventListener("submit", (e) => {
  e.preventDefault();
  const salesBox = $("newSales"), ordersBox = $("newOrders");
  const sales = Number(salesBox.value);
  const orders = Number(ordersBox.value);

  if (!salesBox.value || !ordersBox.value) return message("Enter both sales and orders.", "err");
  if (sales <= 0 || orders <= 0 || !Number.isInteger(sales) || !Number.isInteger(orders)) {
    return message("Use whole numbers greater than zero.", "err");
  }
  if (orders > sales) return message("Orders cannot be more than sales in this dataset.", "err");

  const added = addMonth(sales, orders);
  setRange("all");
  message(`${added} added. Charts and statistics updated.`, "ok");
  e.target.reset();
});

// live feed: a new month every 2.5 seconds, roughly following the trend
let liveTimer = null;
$("liveBtn").addEventListener("click", () => {
  const badge = $("liveBadge");
  if (liveTimer) {
    clearInterval(liveTimer);
    liveTimer = null;
    $("liveBtn").textContent = "Start live feed";
    badge.textContent = "Static data";
    badge.classList.remove("on");
    return;
  }
  setRange("all");
  $("liveBtn").textContent = "Stop live feed";
  badge.textContent = "Live feed";
  badge.classList.add("on");
  liveTimer = setInterval(() => {
    const sales = Math.max(60, Math.round(data.sales.at(-1) + (Math.random() * 60 - 25)));
    const orders = Math.round(sales / (2.3 + Math.random() * 0.3));
    addMonth(sales, orders);
    refresh();
  }, 2500);
});

$("resetBtn").addEventListener("click", () => {
  data = structuredClone(ORIGINAL);
  newRowIndex = -1;
  message("Data reset to the original six months.");
  setRange("all");
});

// ---------- controls ----------
function activate(groupId, btn) {
  document.querySelectorAll(`#${groupId} button`).forEach((b) => b.classList.toggle("active", b ===
btn));
}

function setRange(value) {
  range = value;
  activate("rangeControl", document.querySelector(`#rangeControl [data-range="${value}"]`));
  refresh();
}
```

```
$("rangeControl").addEventListener("click", (e) => {
  const btn = e.target.closest("button");
  if (btn) setRange(btn.dataset.range);
});

$("lineMode").addEventListener("click", (e) => {
  const btn = e.target.closest("button");
  if (!btn) return;
  activate("lineMode", btn);
  lineChart.data.datasets.forEach((d) => (d.tension = Number(btn.dataset.tension)));
  lineChart.update();
});

$("barMode").addEventListener("click", (e) => {
  const btn = e.target.closest("button");
  if (!btn) return;
  activate("barMode", btn);
  barChart.destroy();
  buildBarChart(btn.dataset.mode === "horizontal");
});

// ---------- theme ----------
// Chart.js keeps the colours it was built with, so both charts are rebuilt on a theme change
function rebuildCharts() {
  const tension = Number(document.querySelector("#lineMode .active").dataset.tension);
  const horizontal = document.querySelector("#barMode .active").dataset.mode === "horizontal";
  lineChart.destroy();
  barChart.destroy();
  applyChartDefaults();
  buildLineChart(tension);
  buildBarChart(horizontal);
}

function applyTheme(theme) {
  document.documentElement.dataset.theme = theme;
  $("themeBtn").textContent = theme === "dark" ? "Light mode" : "Dark mode";
}

$("themeBtn").addEventListener("click", () => {
  const next = document.documentElement.dataset.theme === "dark" ? "light" : "dark";
  applyTheme(next);
  try { localStorage.setItem("iwd-theme", next); } catch (err) { /* storage blocked */ }
  rebuildCharts();
});

// ---------- start ----------
try { applyTheme(localStorage.getItem("iwd-theme") || "light"); } catch (err) { applyTheme("light");
}

if (typeof Chart === "undefined") {
  document.querySelector(".charts").innerHTML =
    '<p class="sub" style="padding:40px 0">Chart.js could not be loaded. Check the internet
connection and refresh.</p>';
} else {
  // wait for the web fonts so the chart text is measured with the right font
  document.fonts.ready.then(() => {
    applyChartDefaults();
    buildLineChart();
    buildBarChart();
    renderNumbers();
  });
}
```

The full `style.css` is in the repository. The chart container and responsive rules are shown below.

experiment-3/style.css

```
.charts { display: grid; grid-template-columns: 1fr 1fr; border-bottom: 1px solid var(--ink); }
.chart-block { padding: 34px 40px 30px 0; min-width: 0; }
.chart-block + .chart-block { padding: 34px 0 30px 40px; border-left: 1px solid var(--rule); }
.canvas-wrap { position: relative; height: 340px; }
```

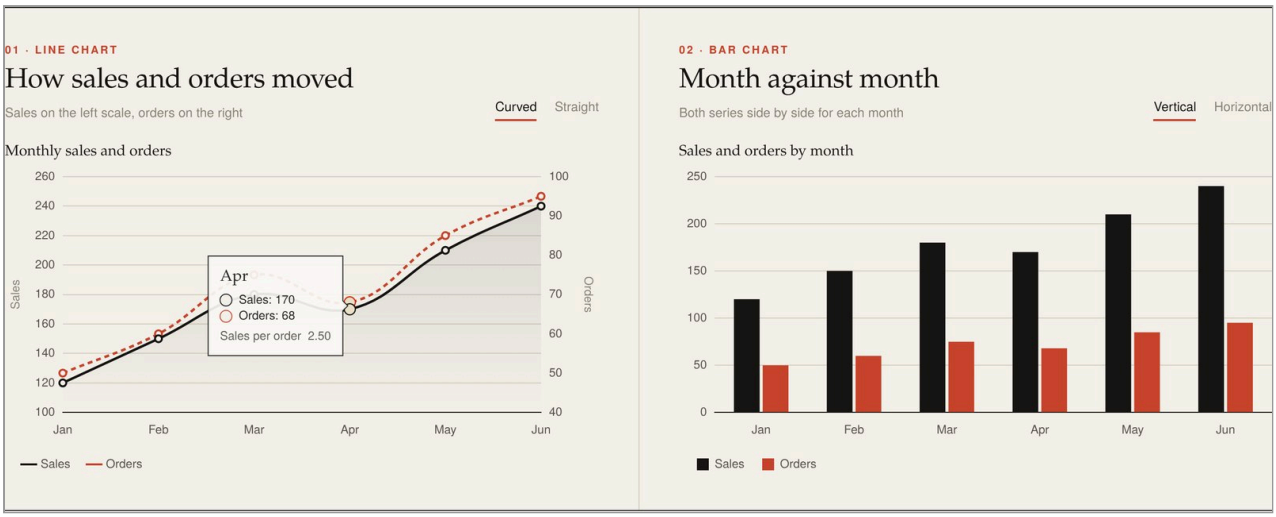
experiment-3/style.css

```
/* ---------- responsive ---------- */
@media (max-width: 1024px) {
  .hero { grid-template-columns: 1fr; gap: 20px; }
  .kpis { grid-template-columns: 1fr 1fr; }
  .kpi:nth-child(3) { border-left: 0; padding-left: 0; }
  .kpi:nth-child(n + 3) { border-top: 1px solid var(--rule); }
  .charts, .tables { grid-template-columns: 1fr; }
  .chart-block, .tables > article { padding-right: 0; }
  .chart-block + .chart-block, .tables > article + article { padding-left: 0; border-left: 0; border-
top: 1px solid var(--rule); }
}
@media (max-width: 560px) {
  .kpi { padding: 18px 0 16px 16px; }
  .canvas-wrap { height: 280px; }
  .block-head h2 { font-size: 23px; }
  .add-row .btn { width: 100%; }
}
```

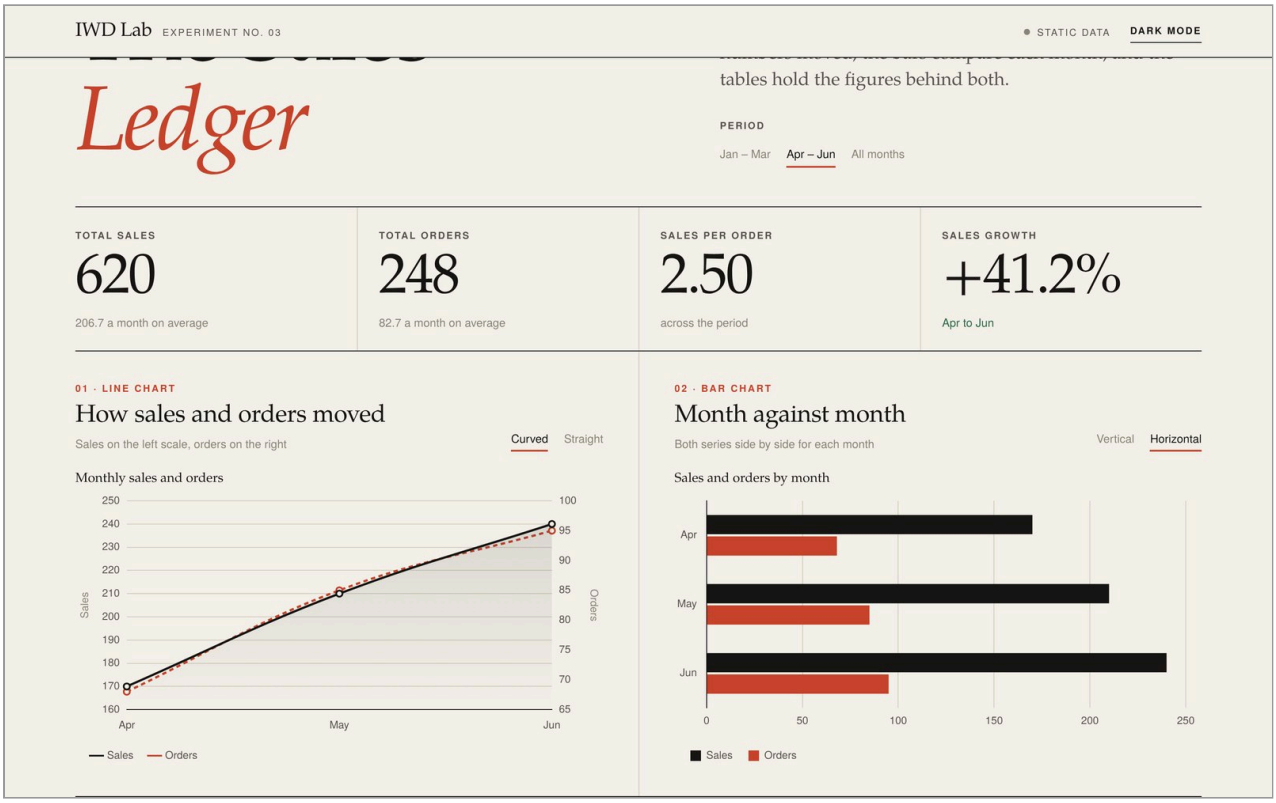
Output



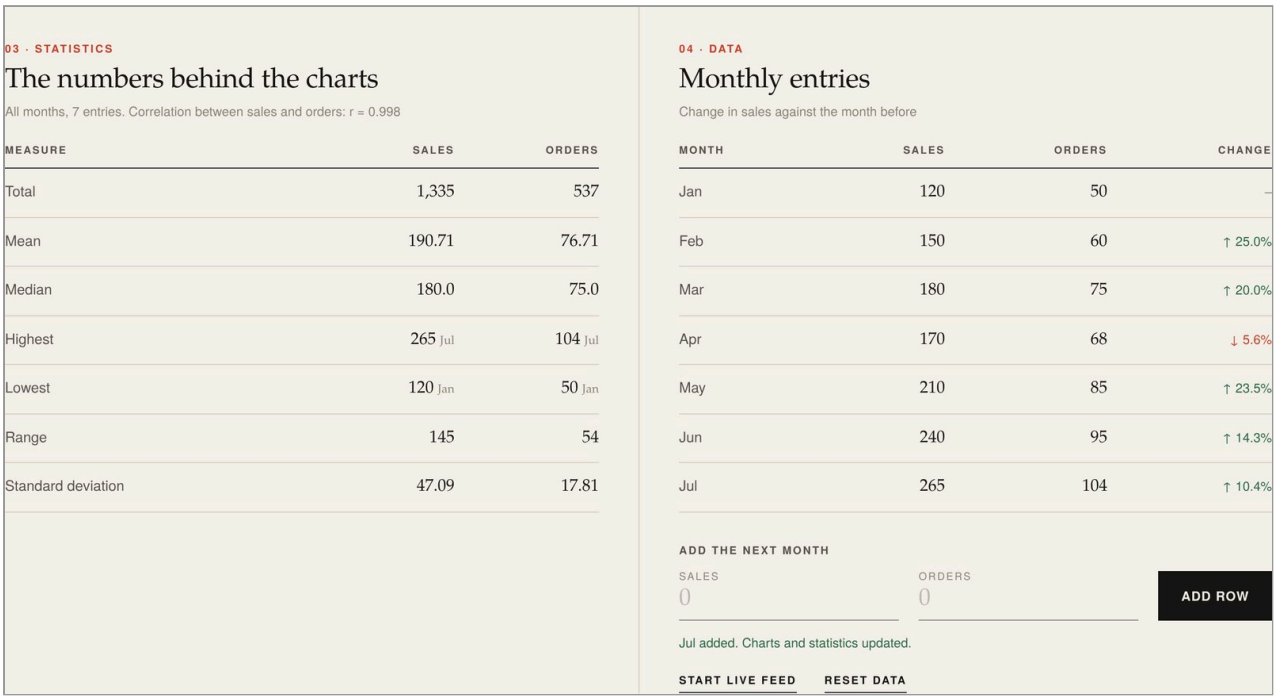
Full dashboard with all six months: headline figures, line chart, bar chart, statistics and monthly entries.



Hovering over April on the line chart shows the tooltip with sales, orders and sales per order.



Period set to Apr to Jun and the bar chart switched to horizontal. The figures now show 620 sales, 248 orders and 41.2% growth.



03 · STATISTICS

The numbers behind the charts

All months, 7 entries. Correlation between sales and orders: r = 0.998

MEASURE	SALES	ORDERS
Total	1,335	537
Mean	190.71	76.71
Median	180.0	75.0
Highest	265 Jul	104 Jul
Lowest	120 Jan	50 Jan
Range	145	54
Standard deviation	47.09	17.81

04 · DATA

Monthly entries

Change in sales against the month before

MONTH	SALES	ORDERS	CHANGE
Jan	120	50	–
Feb	150	60	↑ 25.0%
Mar	180	75	↑ 20.0%
Apr	170	68	↓ 5.6%
May	210	85	↑ 23.5%
Jun	240	95	↑ 14.3%
Jul	265	104	↑ 10.4%

ADD THE NEXT MONTH

SALES 0 ORDERS 0 ADD ROW

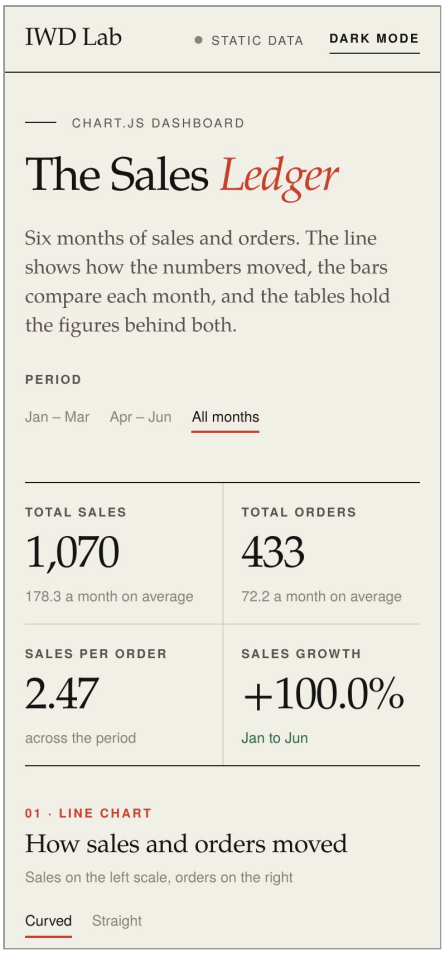
Jul added. Charts and statistics updated.

START LIVE FEED RESET DATA

After adding July (265 sales, 104 orders) the tables and statistics update and the new row is highlighted.



Dark theme after running the live feed. New months (Jul to Sep) were added to both charts automatically.



Dashboard at 390 px (phone) width. The headline figures move to two per row and the charts stack.

Result

The dashboard shows the sales trend in a line chart and the month comparison in a bar chart. Both charts have titles, legends and tooltips, resize with the window and update together with the headline figures and statistics whenever the period changes or new data is added.

Conclusion

I learned how a Chart.js chart is made from a canvas, a data object and an options object, and how to change a chart later by editing `chart.data` and calling `update()`. Using two y-axes made the orders line readable next to sales. Working out the statistics myself also showed that the two series are almost perfectly related, which is easy to see once both lines are on one chart.

GitHub Link

Repository: https://github.com/mahadishaaqlubega/iwd-practicals
Live demo: https://mahadishaaqlubega.github.io/iwd-practicals/